1. Escenario y requerimientos no funcionales

Diseñamos una plataforma SaaS multi-inquilino donde cada emprendedor tiene su propia tienda online. El NFR estrella del escenario 4B es el aislamiento entre tiendas, seguido de la escalabilidad.

Aspecto	Escenario 4B · Plataforma comercial
Escala	5.000 tiendas, con crecimiento esperado a 20.000. Unas 50 tiendas Premium concentran el 40 % del tráfico.
Quién lo mantiene	Un equipo de 12 personas.
Infraestructura	Nube con pago por uso: el costo crece con la cantidad de tiendas, no por adelantado.
NFR estrella	Aislamiento entre tiendas y escalabilidad.
Restricción	Los contratos Premium exigen datos físicamente separados y que su tráfico no afecte ni sea afectado por el resto.
Caso de uso mínimo	Un dueño de tienda carga un producto y un comprador lo ve en esa tienda, y no en otra.

Alcance del TP. Además del caso de uso mínimo, el repositorio implementa el checkout con una pasarela de pagos simulada, las estadísticas de la plataforma, el alta y la baja de tiendas y el cambio de plan entre Estándar y Premium. Quedan fuera las notificaciones, los envíos, las fotos de productos, el dominio propio por tienda y la prueba de carga.

NFRs

NFR	Descripción	Métrica acordada
Aislamiento ⭐	Ningún dueño ve, modifica ni infiere datos de otra tienda. Las Premium no comparten base ni instancias con el resto.	0 accesos cruzados en los tests de punta a punta. Cada tienda Premium corre en su propia celda, con app y base propias.
Escalabilidad ⭐	Crecer de 5.000 a 20.000 tiendas y absorber los picos de las Premium.	Sumar una tienda no requiere cambios de infraestructura; una celda Premium nueva se crea en ejecución, sin cambios de código.

NFR	Descripción	Métrica acordada
Eficiencia de costos	Pago por uso: el costo crece con las tiendas y las transacciones.	Sin infraestructura dedicada para las tiendas estándar; una celda Premium solo existe mientras la tienda tiene el plan.
Mantenibilidad	Operable por 12 personas con automatización.	Un único código para todas las celdas; un comando (`npm test`) levanta el entorno completo y corre los 51 tests.
Disponibilidad	Alta disponibilidad en el flujo de compra.	99,9 % mensual en checkout. Es un objetivo para producción; no se mide en el TP.
Seguridad y privacidad	Identidad estándar, separación física de los datos Premium y ningún dato de tarjeta en la plataforma.	Sin token, 401; con un rol o una tienda que no corresponden, 403. La tarjeta solo la ve la pasarela.

2. Dominio

El dominio es el comercio electrónico multi-inquilino: darle a miles de emprendedores la infraestructura para vender, cada uno en su tienda. El core es lo que usa el emprendedor todos los días y la gestión de planes, porque en 4B el aislamiento por plan es parte del producto que se vende.

Subdominios

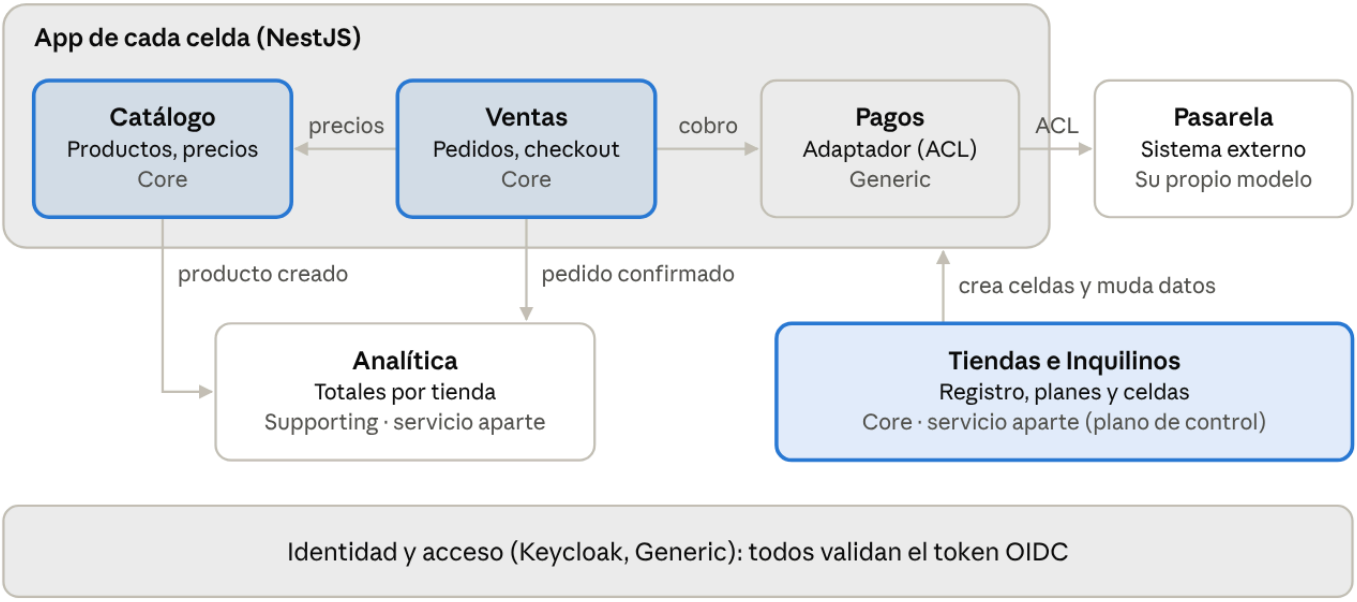
Subdominio	Tipo	Dónde vive en el sistema
Catálogo	Core	Módulo Catálogo de la app de cada celda.
Ventas (checkout y pedidos)	Core	Módulo Ventas de la app de cada celda. El carrito vive en el navegador.
Tiendas e Inquilinos (alta, planes y celdas)	Core	Servicio aparte, el plano de control (ADR-012).
Pagos	Generic	Pasarela externa detrás de un ACL, en el módulo Pagos (ADR-007).
Estadísticas de la plataforma	Supporting	Servicio de Analítica, alimentado por eventos (ADR-006).

Subdominio	Tipo	Dónde vive en el sistema
Identidad y acceso	Generic	Keycloak, proveedor OIDC (ADR-005).
Notificaciones	Supporting	Fuera del alcance del TP.

Bounded contexts y cómo se relacionan

Cada contexto es dueño de sus datos: ninguno lee las tablas de otro. Catálogo, Ventas y Pagos son módulos de la app que corre en cada celda. Tiendas e Inquilinos y Analítica son servicios aparte, porque necesitan una sola instancia para toda la plataforma.

Cada contexto es dueño de sus datos y se comunica por puertos o eventos



Mapa de contextos · 6 contextos, en la celda y como servicios aparte

Relación	Cómo se comunican	Patrón
Ventas ⟶ Catálogo	Ventas pide productos y precios por su puerto Catalogo; un adaptador usa el caso de uso público de Catálogo.	Customer–Supplier
Ventas ⟶ Pagos ⟶ pasarela externa	Ventas cobra por el puerto PasarelaPagos; el adaptador traduce al modelo de la pasarela.	Anti-Corruption Layer

Relación	Cómo se comunican	Patrón
Catálogo y Ventas → Analítica	Publican eventos (producto creado, pedido confirmado) en RabbitMQ; no saben quién escucha.	Published Language, Publish-Subscribe
Tiendas e Inquilinos → gateway	El gateway lee el registro de tiendas y celdas cada 2 segundos.	Open Host Service
Tiendas e Inquilinos → celdas	Crea celdas Premium y muda los datos de una tienda entre celdas.	Plano de control
Todos → Identidad	Validan el token OIDC y sus roles; el dueño trae el claim tienda_id.	Conformist

Modelo táctico: Catálogo

Elemento	Tipo	Detalle
Producto	Agregado raíz	Única puerta de entrada para crear o modificar un producto; siempre pertenece a una tienda.
TiendaId	Value Object	Identidad del inquilino; todo acceso a datos se filtra por él.
Precio	Value Object	Monto y moneda; no puede ser negativo. Se guarda en centavos.
Sku	Value Object	Código único dentro de la tienda.
ProductoRepository	Repositorio (puerto)	Guarda y busca productos sin saber si la base es la del pool o una Premium.

Modelo táctico: Ventas

Elemento	Tipo	Detalle
Pedido	Agregado raíz	Un intento de compra en una tienda. El total sale de los precios de la tienda, en una sola moneda; el cliente solo elige productos y cantidades. El resultado del cobro se registra una sola vez.

Elemento	Tipo	Detalle
LineaPedido	Entidad	Producto, cantidad y precio copiados al confirmar: no cambian aunque después cambie el catálogo.
EstadoPedido	Value Object	Pendiente, Pagado o Rechazado.
DatosComprador y DatosEnvío	Value Objects	Nombre, email y teléfono de quien compra; dirección, ciudad y código postal.
PedidoRepository	Repositorio (puerto)	Guarda y busca pedidos, siempre dentro de la tienda.
Catalogo y PasarelaPagos	Puertos	Lo que Ventas necesita de afuera: precios y cobro.

Modelo táctico: Tiendas e Inquilinos

Elemento	Tipo	Detalle
Tienda	Agregado raíz	Subdominio, nombre, plan, celda donde vive y estado (activa o mudándose).
Plan	Value Object	Estándar o Premium.
Celda	Entidad	Donde corren las tiendas: la pool compartida o una celda Premium. Conoce la dirección de su app y de su base.
SolicitudPremium	Entidad	El pedido de un dueño para cambiar de plan: pendiente, en curso, completada, fallida o rechazada, con el paso de la mudanza.
Registro	Repositorio	La única fuente de verdad de tiendas y celdas.

Reglas de negocio

1. **Aislamiento:** un emprendedor nunca puede ver, modificar ni inferir datos de otra tienda.
2. **Estadísticas consolidadas:** la plataforma junta totales por tienda (productos publicados, pedidos confirmados y monto vendido) para uso exclusivo del administrador, que los usa para decidir qué tiendas conviene pasar a Premium. Nunca guarda datos de compradores ni el detalle de productos o pedidos.
3. **Caso de uso mínimo:** un dueño carga un producto en su panel y un comprador solo puede verlo dentro de esa tienda.

4. **Alta de tiendas:** solo el administrador da de alta tiendas. Toda tienda nueva empieza en el pool, con plan Estándar, y su dueño queda listo para ingresar.
5. **Cambio de plan:** el dueño pide pasar a Premium, o volver a Estándar, y el administrador lo aprueba. La tienda se muda de celda con todos sus datos, sin pasos manuales; mientras dura la mudanza se puede ver pero no acepta compras ni cambios.

3. Elección del estilo: Matriz de Pugh

Elegimos un **modelo híbrido: pool compartido para las tiendas estándar y una celda dedicada por tienda Premium**, con el mismo monolito modular desplegado en cada celda. Es la única alternativa que cumple la restricción Premium sin romper el costo por uso ni la capacidad del equipo.

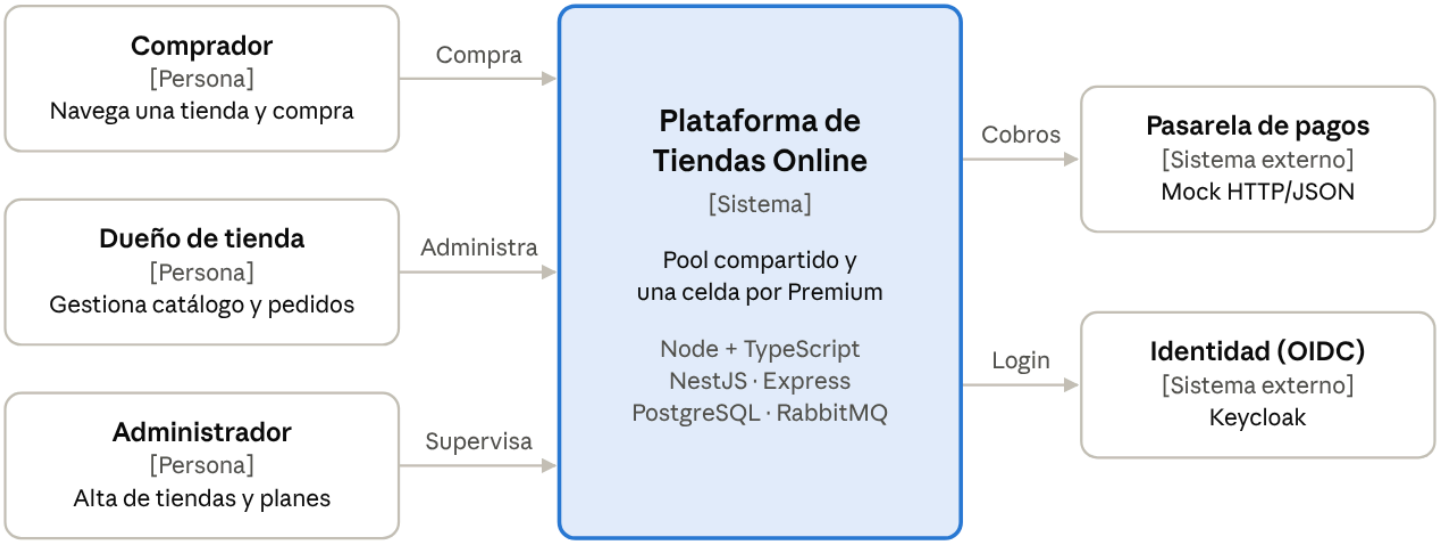
El baseline es la solución natural del escenario 4A: monolito modular con una base compartida y tienda_id.

Criterio	Peso	Baseline	A1 Microservicios en pool	A2 Silo para todos	A3 Híbrido pool + silo
Aislamiento Premium	×2	0	0	+1	+1
Escalabilidad elástica	×2	0	+1	0	+1
Costo proporcional al uso	×1	0	−1	−1	0
Carga operativa (12 personas)	×1	0	−1	−1	−1
Time-to-market	×1	0	−1	−1	−1
Total simple		**0**	**−2**	**−2**	**0**
Total ponderado		**0**	**−1**	**−1**	**+2**
Cumple la restricción Premium		No	No	Sí	Sí

Sin ponderar, A3 empata con el baseline. Pero el aislamiento físico es una restricción, no un criterio más: no se optimiza, se respeta. El baseline y A1 quedan descalificados antes de sumar puntos. El detalle de cada puntaje está en el documento de la Matriz de Pugh.

4. C4: Contexto

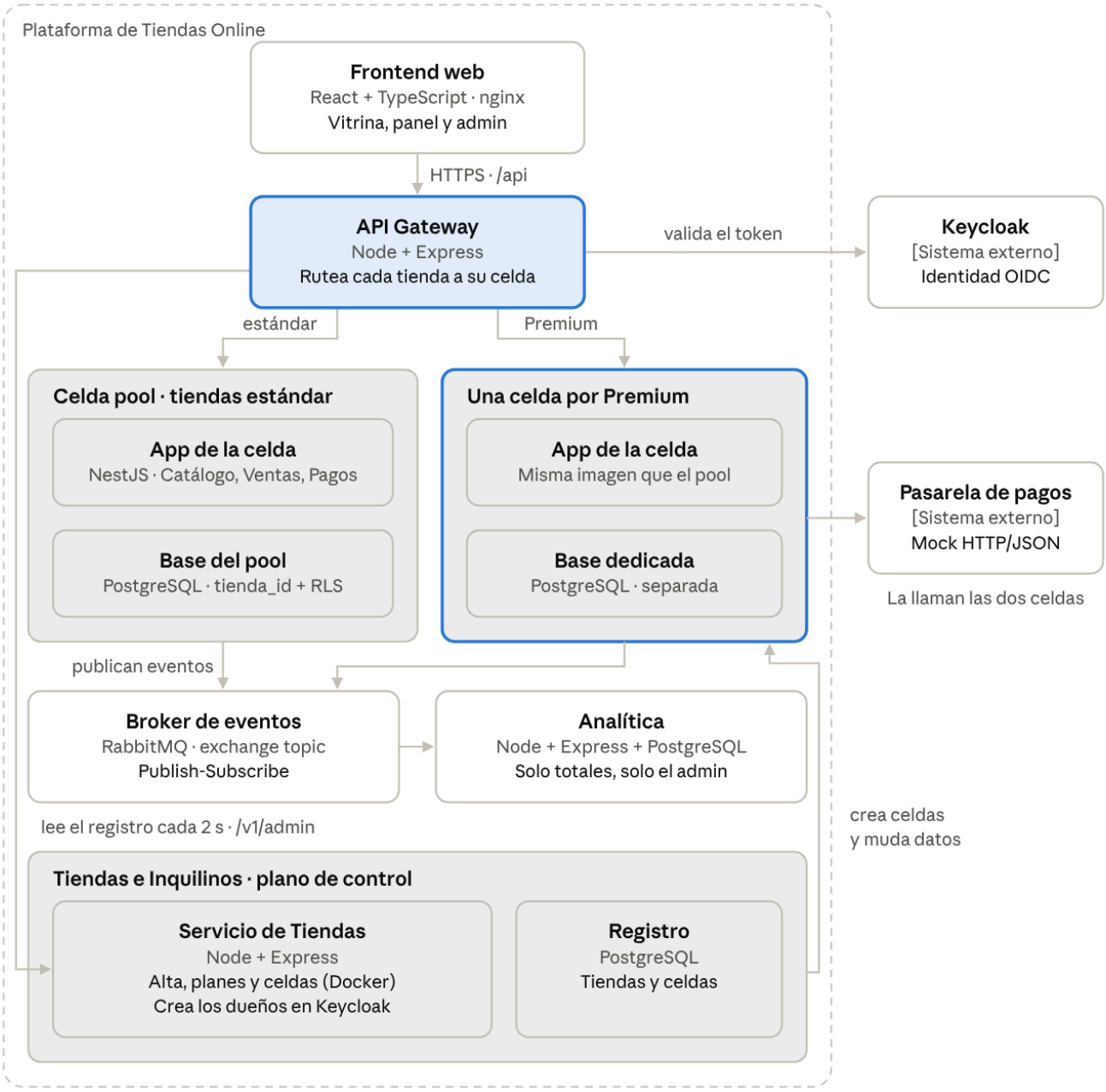
Tres tipos de usuario y dos sistemas externos



La plataforma atiende a compradores, dueños de tienda y al administrador, y depende de dos sistemas externos: la pasarela de pagos (simulada con un mock HTTP/JSON) y el proveedor de identidad OIDC (Keycloak). El administrador no pertenece a ninguna tienda: da de alta tiendas, aprueba los cambios de plan y es el único que ve las estadísticas consolidadas.

5. C4: Contenedores

Una celda por tienda Premium; el resto comparte el pool



El frontend llama a la API en su mismo origen y nginx la reenvía al gateway. El gateway valida el token contra Keycloak, resuelve la tienda por el subdominio y rutea cada request a su celda, según el registro que le da el servicio de Tiendas e Inquilinos. Todas las celdas corren la misma

imagen de la app, cada una con su base PostgreSQL 16; en el pool la base es compartida y se aísla con tienda_id y row-level security.

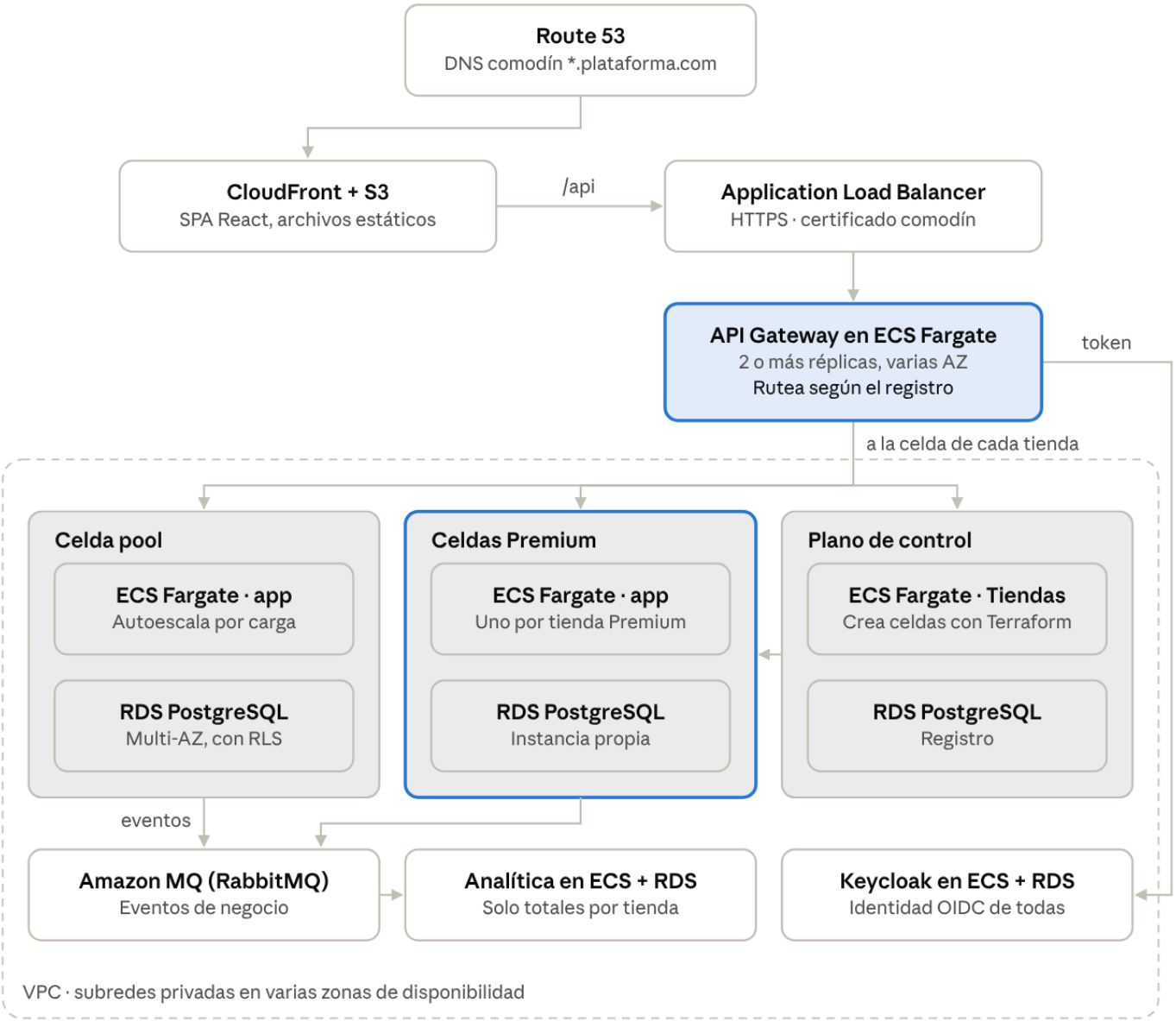
Las celdas publican eventos en RabbitMQ y Analítica guarda solo totales por tienda; el gateway le pasa las consultas de /v1/admin/estadisticas. El docker compose levanta el pool y una celda Premium de ejemplo (premium-a); las demás las crea el servicio de Tiendas al aprobar un pase a Premium.

Contenedor	Tecnología	Responsabilidad
Frontend web	React + TypeScript (Vite), servido por nginx	Vitrina de cada tienda, panel del dueño y administración de la plataforma
API Gateway	Node + Express	Valida el token, resuelve la tienda por subdominio, rutea a su celda y frena escrituras durante una mudanza
App de la celda	NestJS + TypeScript, Sequelize	Módulos Catálogo, Ventas y Pagos; una instancia por celda
Base de la celda	PostgreSQL 16	Datos de las tiendas de la celda, con row-level security
Servicio de Tiendas e Inquilinos	Node + Express + PostgreSQL 16	Registro, alta y baja de tiendas, cambio de plan, creación de celdas
Broker de eventos	RabbitMQ 3.13	Exchange topic eventos
Analítica	Node + Express + PostgreSQL 16	Totales por tienda para el administrador
Keycloak	Keycloak 23	Identidad OIDC, roles y claim tienda_id
Pasarela simulada	Node + Express	Cobros con Idempotency-Key

6. Vista de despliegue

En el TP, docker compose levanta todo en una máquina: cada celda es un par de contenedores (app y base) y las celdas Premium nuevas las crea el servicio de Tiendas con la API de Docker. En producción, cada celda sería un servicio de ECS con su propia base en RDS, creado por Terraform.

En AWS, cada celda es un servicio de ECS con su propia base



Despliegue propuesto en AWS · pool, celdas Premium y plano de control

El gateway y el plano de control son únicos para toda la plataforma; lo que se multiplica con las Premium son las celdas. El costo de una celda Premium lo paga la tienda que tiene ese plan.

Componente	En el TP (docker compose)	En AWS
Dirección de cada tienda	`*.localhost` lo resuelve el navegador	Route 53 con un DNS comodín y un certificado comodín de ACM

Componente	En el TP (docker compose)	En AWS
Frontend web	nginx en un contenedor	S3 + CloudFront
API Gateway	Un contenedor Express	ECS Fargate detrás de un Application Load Balancer, con 2 o más réplicas en varias zonas
App de cada celda	Un contenedor por celda	Un servicio de ECS Fargate por celda, con autoescalado
Base de cada celda	Un contenedor PostgreSQL por celda	RDS for PostgreSQL por celda; la del pool, Multi-AZ
Creación de celdas Premium	API de Docker desde el servicio de Tiendas	Terraform (o CloudFormation) disparado por el servicio de Tiendas, con un rol limitado
Tiendas e Inquilinos	Contenedor Express + PostgreSQL	ECS Fargate + RDS
Broker de eventos	Contenedor RabbitMQ	Amazon MQ for RabbitMQ
Analítica	Contenedor Express + PostgreSQL	ECS Fargate + RDS
Identidad	Contenedor Keycloak	Keycloak en ECS + RDS, o Amazon Cognito
Credenciales de las bases	Variables de entorno	Secrets Manager
Logs y métricas	`docker compose logs`	CloudWatch
Imágenes y despliegue	Build local	GitHub Actions publica en ECR y despliega en todas las celdas del registro

7. Flujos clave

Tres flujos muestran las decisiones en funcionamiento: el caso de uso mínimo prueba el aislamiento, el checkout la idempotencia y el pase a Premium el plano de control.

Caso de uso mínimo: cargar un producto y verlo solo en su tienda

1. El dueño entra a la-yerbera.localhost/panel e ingresa con Keycloak; su token trae el rol duenio y tienda_id = t1.
2. El frontend envía POST /api/v1/productos; nginx lo reenvía al gateway con el mismo host.

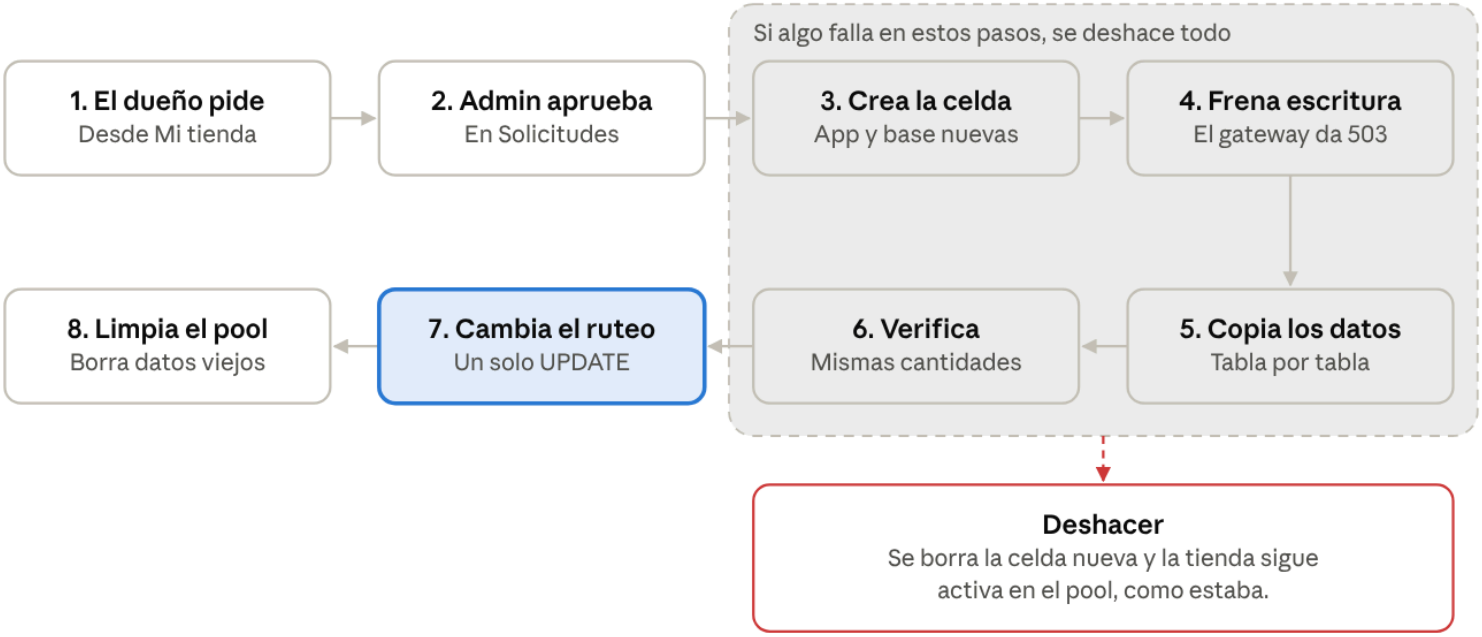
3. El gateway valida el token, resuelve la tienda t1 por el subdominio, traduce la ruta a /v1/tiendas/t1/productos y la manda a la celda pool.
4. En la app, el caso de uso CrearProducto verifica que el tienda_id del token sea t1, crea el Producto y lo guarda con la tienda activa fijada: la base solo acepta filas de t1. Después publica "producto creado".
5. Un comprador en ceramica-nandu.localhost pide /v1/productos: el gateway lo traduce a la tienda t2 y la base, por row-level security, no devuelve el producto de t1.

Checkout idempotente

1. El navegador genera una Idempotency-Key para el intento de compra y envía los productos, las cantidades y los datos de entrega, sin precios.
2. Ventas busca un pedido con esa clave. Si existe con el mismo carrito, lo devuelve tal cual (200), sin cobrar de nuevo; si existe con otro carrito, responde 422.
3. Si no existe, le pide los precios a Catálogo, arma el Pedido pendiente y lo guarda.
4. Cobra a la pasarela a través del ACL, con el id del pedido como clave. Si la pasarela no responde en 5 segundos, el pedido queda pendiente y se puede reintentar sin riesgo.
5. Registra el resultado (pagado o rechazado). Si se pagó, publica "pedido confirmado" para Analítica.

Pase a Premium

Pase a Premium: el ruteo cambia recién cuando la copia está verificada



Pase a Premium · 8 pasos, los del 3 al 6 reversibles

La solicitud guarda el paso en el que está, así el administrador sigue la mudanza en pantalla. El cambio de ruteo es un solo UPDATE en el registro: el gateway nunca ve una tienda a medio mudar. Si el servicio se reinicia en medio, al arrancar termina la mudanza (si el ruteo ya había cambiado) o la deshace. La vuelta a Estándar es el mismo circuito hacia el pool, y al final se borra la celda Premium de la tienda.

8. Decisiones de arquitectura (ADRs)

Los trece ADR están aceptados e implementados; el texto completo de cada uno está en `docs/adr/` del repositorio.

ADR	Decisión	Favorece	Perjudica
001	Tenancy híbrido: pool + silo por Premium	Aislamiento, escalabilidad, costo	Mantenibilidad
002	Monolito modular desplegado por celdas	Mantenibilidad, time-to-market	Escala por celda, no por módulo
003	tienda_id + row-level security en el pool	Seguridad, aislamiento	Complejidad en la capa de datos
004	Ruteo de tenants en el API Gateway	Escalabilidad, aislamiento	El gateway es punto único de falla
005	Identidad con OIDC (Keycloak)	Seguridad, mantenibilidad	El login sale del sitio
006	Estadísticas por eventos (Publish-Subscribe)	Aislamiento, escalabilidad	Un broker y un servicio más
007	Pasarela detrás de un ACL, con idempotencia	Seguridad, disponibilidad	—
008	Node + TypeScript (NestJS) y PostgreSQL	Mantenibilidad, time-to-market	—
009	La tienda se identifica en la URL de la API	Aislamiento, simplicidad	—
010	Frontend como SPA en React	Usabilidad, mantenibilidad	SEO limitado

ADR	Decisión	Favorece	Perjudica
011	Cada tienda en su propio subdominio	Aislamiento, escalabilidad	Más responsabilidad en el gateway
012	Tiendas e Inquilinos, servicio aparte (plano de control)	Aislamiento, escalabilidad operativa	Seguridad del entorno local, escritura durante la mudanza
013	Sequelize como ORM de la app; el esquema sigue en SQL	Mantenibilidad, aislamiento	Una dependencia más en la capa de datos

ADR-001: Modelo de tenancy híbrido (pool + silo Premium)

- **Estado:** Aceptada
- **Contexto:** entre 5.000 y 20.000 tiendas con pago por uso, y contratos Premium que exigen separación física de datos y de tráfico.
- **Decisión:** vamos a alojar las tiendas estándar en un pool compartido y cada tienda Premium en un silo dedicado (instancia de la aplicación + base propia).
- **Alternativas:** pool para todas (no cumple la restricción); silo para todas (costo fijo por tienda, aunque no venda, e inoperable para 12 personas).
- **Consecuencias:** se cumple el contrato Premium y el costo del silo lo paga quien lo usa. Hay que aprovisionar silos y mudar datos entre bases; lo hace automáticamente el servicio de Tiendas e Inquilinos (ADR-012).
- **NFRs:** favorece aislamiento, escalabilidad y costo; perjudica mantenibilidad.

ADR-002: Estilo monolito modular desplegado por celdas

- **Estado:** Aceptada
- **Contexto:** hace falta aislar celdas sin multiplicar el trabajo de un equipo de 12 personas.
- **Decisión:** vamos a construir un único monolito modular (un módulo por bounded context: Catálogo, Ventas, Pagos) y desplegar el mismo artefacto en cada celda: una celda pool y una por cada tienda Premium.
- **Alternativas:** microservicios (más repositorios, pipelines y contratos internos de los que el equipo puede sostener; Ley de Conway); monolito en capas (peor cohesión por dominio).
- **Consecuencias:**
    - Un solo código y un solo pipeline para todas las celdas; los módulos pueden extraerse a servicios si un contexto lo justifica.

    - Tiendas e Inquilinos (ADR-012) y Analítica (ADR-006) no son módulos de la app sino servicios aparte: necesitan una sola instancia para toda la plataforma, y la app corre una vez por celda.
- **NFRs:** favorece mantenibilidad y time-to-market; la escalabilidad es por celda, no por módulo.

ADR-003: Aislamiento de datos en el pool con tienda_id y row-level security

- **Estado:** Aceptada
- **Contexto:** miles de tiendas estándar comparten una base; un filtro olvidado en una consulta expondría datos de otra tienda.
- **Decisión:** vamos a guardar tienda_id en cada tabla y activar row-level security en PostgreSQL. Cada transacción fija la tienda activa y la base filtra por ella. La app se conecta con un usuario que no es dueño de las tablas, así la regla aplica siempre.
- **Alternativas:** solo filtro en el código (un error de programación lo rompe); un esquema por tienda (miles de esquemas que migrar).
- **Consecuencias:** doble barrera de aislamiento, el código y la base. Toda consulta corre con la tienda activa fijada. El servicio de Tiendas también pasa por esta regla al mudar datos: su usuario solo ve la tienda que está mudando. Desde el ADR-013, una guarda sobre los modelos de Sequelize rechaza cualquier consulta que no use la tienda activa.
- **NFRs:** favorece seguridad y aislamiento; agrega algo de complejidad en la capa de datos.

ADR-004: Ruteo de tenants en el API Gateway

- **Estado:** Aceptada
- **Contexto:** cada request tiene que llegar a la celda correcta (pool o silo Premium) sin que el cliente lo sepa.
- **Decisión:** vamos a resolver la tienda en el API Gateway (Node + Express) a partir del subdominio (ADR-011) y rutear a la celda que figura en el registro de tiendas.
- **Alternativas:** que cada cliente conozca la URL de su celda (acopla al cliente con la infraestructura).
- **Consecuencias:**
    - Mover una tienda a Premium es cambiar un registro.
    - El registro lo provee el servicio de Tiendas e Inquilinos (ADR-012). El gateway lo lee cada 2 segundos y, si el servicio no responde, sigue ruteando con la última copia buena.
    - Mientras una tienda se muda de celda, el gateway rechaza sus escrituras con 503.
    - El gateway es un componente crítico y debe estar replicado.
- **NFRs:** favorece escalabilidad y aislamiento; el gateway es un punto único de falla a mitigar.

ADR-005: Identidad con OIDC (Keycloak)

- **Estado:** Aceptada
- **Contexto:** el TP exige OIDC con al menos 2 roles; hay tres: administrador, dueño de tienda y comprador.
- **Decisión:** vamos a usar Keycloak como proveedor OIDC, con roles por usuario y un claim tienda_id en el token de cada dueño. El login se hace en la pantalla de Keycloak (Authorization Code + PKCE), con un tema propio con el diseño del sitio.
- **Alternativas:** autenticación propia (reinventar un subdominio genérico); proveedor en la nube (no corre en el docker compose); formulario de login propio en la SPA (la SPA vería la contraseña, que es lo que OIDC evita; ese flujo se elimina en OAuth 2.1).
- **Consecuencias:** login estándar y desacoplado; la contraseña la ve únicamente Keycloak. El rol lo valida la app; que el dueño opere solo sobre su tienda (claim tienda_id) lo valida el caso de uso, sin depender de HTTP.
- **NFRs:** favorece seguridad y mantenibilidad.

ADR-006: Estadísticas de plataforma por eventos (Publish-Subscribe)

- **Estado:** Aceptada
- **Contexto:** la regla 2 pide totales de todas las tiendas, pero las Premium están en bases físicamente separadas, y consultar todas las bases rompería el aislamiento.
- **Decisión:** vamos a publicar eventos de negocio desde cada celda a un broker (RabbitMQ). Un servicio de Analítica, con su propia base, los consume y guarda solo totales por tienda, visibles únicamente para el administrador.
    - Las celdas publican por un puerto (PublicadorDeEventos) en un exchange de tipo topic: Catálogo publica "producto creado" y Ventas, "pedido confirmado" cuando la pasarela aprueba el pago.
    - Analítica confirma cada mensaje recién después de guardarlo; si llega repetido, lo descarta por su identificador.
- **Alternativas:** consultar todas las bases desde un reporte (rompe el aislamiento y acopla al esquema de cada celda); un módulo de Analítica dentro de la app (los totales quedaban repartidos entre las celdas y había que saltar la seguridad por fila).
- **Consecuencias:**
    - Las celdas no conocen a Analítica: otro servicio puede escuchar los mismos eventos con su propia cola.
    - Consistencia eventual: un producto nuevo aparece en los totales unos instantes después.
    - Si el broker se cae, las tiendas siguen funcionando y los eventos se reintentan. Si una celda se reinicia antes de publicar, ese evento se pierde; un outbox lo evitaría y queda como mejora.
- **NFRs:** favorece aislamiento y escalabilidad; agrega un broker y un servicio más que operar.

ADR-007: Integración con la pasarela de pagos mediante Anti-Corruption Layer

- **Estado:** Aceptada
- **Contexto:** la pasarela es externa, su modelo no es el nuestro y un reintento no puede cobrar dos veces.
- **Decisión:** vamos a aislar la pasarela detrás de un ACL que traduce su modelo, enviar cada cobro con Idempotency-Key y no almacenar datos de tarjeta (tokenización en la pasarela).
    - El checkout es idempotente: el cliente manda una clave por intento de compra. Un reintento con la misma clave devuelve el mismo pedido sin un segundo cobro; la misma clave con otro carrito da error.
    - A la pasarela se le cobra con el id del pedido como clave, así un reintento nunca cobra dos veces.
    - Si la pasarela no responde en 5 segundos, el pedido queda pendiente y se puede reintentar sin riesgo.
- **Alternativas:** llamar a la pasarela directo desde Ventas (su vocabulario contamina el dominio).
- **Consecuencias:** cambiar de pasarela solo toca el adaptador; el alcance PCI-DSS queda reducido.
- **NFRs:** favorece seguridad, disponibilidad y mantenibilidad.

ADR-008: Stack Node + TypeScript (NestJS) y PostgreSQL

- **Estado:** Aceptada
- **Contexto:** hay que elegir el lenguaje y el framework del backend, y una base que soporte el aislamiento del ADR-003.
- **Decisión:** vamos a usar Node + TypeScript con NestJS para la app de las celdas y PostgreSQL 16 como base de cada celda. Los servicios chicos (gateway, Analítica y Tiendas e Inquilinos) usan Node + Express: tienen pocos endpoints y no necesitan la estructura de módulos de NestJS, pero comparten lenguaje y herramientas con la app. En la app, el acceso a datos es con Sequelize (ADR-013).
- **Alternativas:** Spring Boot (Java), .NET y FastAPI (Python).
- **Consecuencias:** NestJS organiza el código en módulos con inyección de dependencias, lo que encaja con el monolito modular y con los puertos y adaptadores de cada módulo. PostgreSQL ofrece row-level security nativo. Un solo lenguaje, TypeScript, en todo el sistema, también en el frontend.
- **NFRs:** favorece mantenibilidad y time-to-market.

ADR-009: La tienda se identifica en la URL de la API

- **Estado:** Aceptada
- **Contexto:** el gateway necesita saber a qué tienda va cada request para elegir la celda. Los compradores navegan sin estar logueados, así que no siempre hay token.

- **Decisión:** vamos a identificar la tienda en el path de la API (/v1/tiendas/{tiendaId}/…).
- **Alternativas:** por claim del token (los compradores anónimos no tienen token); por subdominio (es la dirección pública, ver ADR-011, pero no tiene por qué ser el contrato interno de la API).
- **Consecuencias:** funciona igual para compradores anónimos y para dueños. La app verifica en el caso de uso que el tienda_id del token del dueño coincida con el de la URL. El gateway traduce el subdominio a este path sin cambiar la app.
- **NFRs:** favorece aislamiento y simplicidad.

ADR-010: Frontend como SPA en React

- **Estado:** Aceptada
- **Contexto:** compradores, dueños y administrador necesitan una interfaz web; la lógica de negocio vive en la API, detrás del gateway.
- **Decisión:** vamos a construir una SPA en React + TypeScript (Vite), servida por nginx, que consume la API del gateway y hace login OIDC contra Keycloak con Authorization Code + PKCE.
- **Alternativas:** MPA con render en el servidor (acopla la interfaz al backend y habría que renderizar en cada celda); app móvil (fuera del alcance).
- **Consecuencias:**
    - La interfaz no tiene reglas de negocio: solo muestra datos y captura acciones.
    - Un mismo build sirve la plataforma (presentación y administración en /admin) y cada tienda (vitrina en / y panel del dueño en /panel), según el host.
    - La SPA llama a la API en su mismo origen (/api) y nginx la reenvía al gateway, sin CORS.
    - El comprador no ve conceptos de la arquitectura; la celda que atendió viaja en el header X-Celda.
- **NFRs:** favorece usabilidad y mantenibilidad; SEO limitado, aceptable para el TP.

ADR-011: Cada tienda se publica en su propio subdominio

- **Estado:** Aceptada
- **Contexto:** cada emprendedor quiere una dirección propia para compartir con sus clientes, y no que esos clientes naveguen hacia otras tiendas, que pueden ser su competencia.
- **Decisión:** vamos a publicar cada tienda en su propio subdominio ({subdominio}.plataforma.com; en local, la-yerbera.localhost:5173). El gateway resuelve la tienda a partir del host y, desde el subdominio de una tienda, solo se llega a esa tienda. El panel del dueño está dentro de su tienda (/panel). La raíz de la plataforma es una presentación, sin listado de tiendas.

- **Alternativas:** tienda en el path (la plataforma se ve como un shopping); dominio propio por tienda (DNS y certificado por cada uno; se puede sumar después); panel en un subdominio aparte (el dueño administraría desde otra dirección).
- **Consecuencias:** cada tienda es un origen distinto para el navegador, así que sesión, carrito y almacenamiento quedan separados. Sumar una tienda no requiere cambios de infraestructura: alcanza con un DNS y un certificado comodín. Cada subdominio tiene que estar registrado como destino de login en Keycloak, y el alta lo hace sola.
- **NFRs:** favorece el aislamiento y la escalabilidad; agrega una responsabilidad al gateway.

ADR-012: Tiendas e Inquilinos, un servicio aparte que es el plano de control

- **Estado:** Aceptada
- **Contexto:** el registro de tiendas y celdas tiene que ser uno solo, porque de él depende el ruteo, y la app corre una vez por celda. Además, el administrador tiene que poder dar de alta tiendas y el dueño, cambiar de plan, sin pasos manuales.
- **Decisión:** vamos a tener un servicio de Tiendas e Inquilinos (Node + Express), aparte de las celdas y con su propia base.
    - **Registro:** las tiendas y las celdas son la única fuente de verdad; el gateway las lee cada 2 segundos.
    - **Alta y baja** (solo el administrador): tocan el registro, Keycloak y la celda sin una transacción común. Si falla un paso, se deshacen los anteriores (compensación).
    - **Cambio de plan:** el dueño lo pide, el administrador lo aprueba y el servicio muda la tienda a una celda nueva, o de vuelta al pool (sección 7). La copia de datos pasa por row-level security.
    - **Celdas nuevas:** se crean con la API de Docker, clonando la celda pool (misma imagen, mismos scripts de la base, misma red). En producción lo haría Terraform contra la nube.
- **Alternativas:** un módulo Tiendas dentro de la app (el registro quedaría repetido en cada celda); celdas Premium ya levantadas esperando (se paga infraestructura que nadie usa); copia en vivo sin frenar escrituras (CDC o doble escritura: mucha complejidad para algo que pasa una vez por tienda).
- **Consecuencias:**
    - Alta, baja y cambio de plan son automáticos.
    - Si el servicio se cae, las tiendas siguen funcionando con la última copia del registro; solo no se pueden hacer cambios.
    - Durante la mudanza hay un corte de escrituras de unos 15 a 20 segundos más el tiempo de copia; la tienda se sigue viendo.
    - Montar el socket de Docker da acceso total al Docker del host: se acepta en la simulación local; en producción el servicio usaría la API de la nube con un rol limitado.

- **NFRs:** favorece aislamiento, escalabilidad operativa y disponibilidad del ruteo; perjudica la seguridad del entorno local y la escritura de la tienda que se muda, durante unos segundos.

ADR-013: Sequelize como ORM de la app

- **Estado:** Aceptada
- **Contexto:** Catálogo y Ventas accedían a la base con pg y SQL escrito a mano en cada repositorio. Además, que toda consulta usara la tienda activa (ADR-003) dependía de la disciplina: una consulta olvidada devolvía cero filas sin error.
- **Decisión:** vamos a usar Sequelize 6 en la capa de infraestructura de la app.
    - **Puertos y adaptadores:** el dominio no cambia; los repositorios de Sequelize implementan los mismos puertos, con un mapper por módulo.
    - **El esquema sigue en SQL:** tablas, usuarios y políticas de row-level security los crea el script de la base. Sequelize solo lee y escribe.
    - **Guarda:** cada modelo rechaza cualquier operación fuera de la transacción con la tienda activa.
    - **Alcance:** solo la app. Tiendas y Analítica siguen con pg porque tienen pocas consultas y son fijas.
- **Alternativas:** seguir con pg (no hay forma simple de exigir la tienda activa); TypeORM (sus entidades con decoradores tienden a mezclarse con el dominio); Prisma (su esquema sería una segunda fuente de verdad y no expresa row-level security); que Sequelize cree el esquema (no modifica tablas existentes y el servicio de Tiendas necesita el script para crear celdas).
- **Consecuencias:**
    - Cada consulta tiene que pasar la transacción; si no, la guarda lanza un error.
    - Un cambio de esquema se hace en el SQL y en el modelo a la vez. Todavía no hay migraciones versionadas.
    - Cambiar de ORM solo toca la infraestructura de cada módulo.
- **NFRs:** favorece mantenibilidad y aislamiento; agrega una dependencia más.

9. Verificación de NFRs

Ocho de los nueve puntos están verificados con tests ejecutables; la prueba de carga no se llegó a hacer. `npm test` levanta toda la plataforma con docker compose y corre 51 tests de punta a punta contra el gateway.

NFR	Cómo se verifica	Tests	Resultado
Aislamiento de datos	El dueño de una tienda intenta cargar productos y ver pedidos de otra y recibe 403. Comprar desde una tienda un producto de otra da	`aislamiento.test.js` (11),	Verificado: 0 accesos cruzados

NFR	Cómo se verifica	Tests	Resultado
	404. Lo que se carga en una tienda no aparece en otra, aunque compartan la base del pool.	`pedidos.test.js` (16)	
Separación física Premium	Cada respuesta dice qué celda la atendió (header X-Celda). Una tienda pasa a Premium, se muda a una celda nueva con su propia base y sus productos y pedidos siguen estando; al volver a Estándar, la celda se borra.	`premium.test.js` (11)	Verificado
Escalabilidad	Se crea una celda Premium nueva en ejecución, sin cambios de código ni reinicios. Dar de alta una tienda no requiere cambios de infraestructura.	`premium.test.js`, `tiendas.test.js` (8)	Verificado
Mantenibilidad	Un solo comando levanta la plataforma y corre los tests; el CI compila los 6 servicios en cada push.	Todos	Verificado
Seguridad	Login OIDC con 3 roles. Sin token, 401; con un rol o una tienda que no corresponden, 403.	Todos	Verificado
Pagos sin duplicar	Reintentar un pedido con la misma Idempotency-Key devuelve el mismo pedido y el mismo cobro; la misma clave con otro carrito da error.	`pedidos.test.js`	Verificado
Estadísticas sin romper el aislamiento	Cargar un producto o pagar un pedido suma en los totales de su tienda; un pago rechazado o un reintento no suman. Solo el administrador las ve.	`estadisticas.test.js` (5)	Verificado
Datos personales de compradores	Los pedidos, con los datos del comprador, solo los ve el dueño de esa tienda: sin token 401; otro dueño o un comprador, 403.	`pedidos.test.js`	Verificado

NFR	Cómo se verifica	Tests	Resultado
Aislamiento de recursos	Prueba de carga sobre el pool midiendo la latencia de una tienda Premium.	—	No realizado. Se apoya en la estructura: cada Premium tiene su propia app y su propia base, y con el pool solo comparte el gateway.